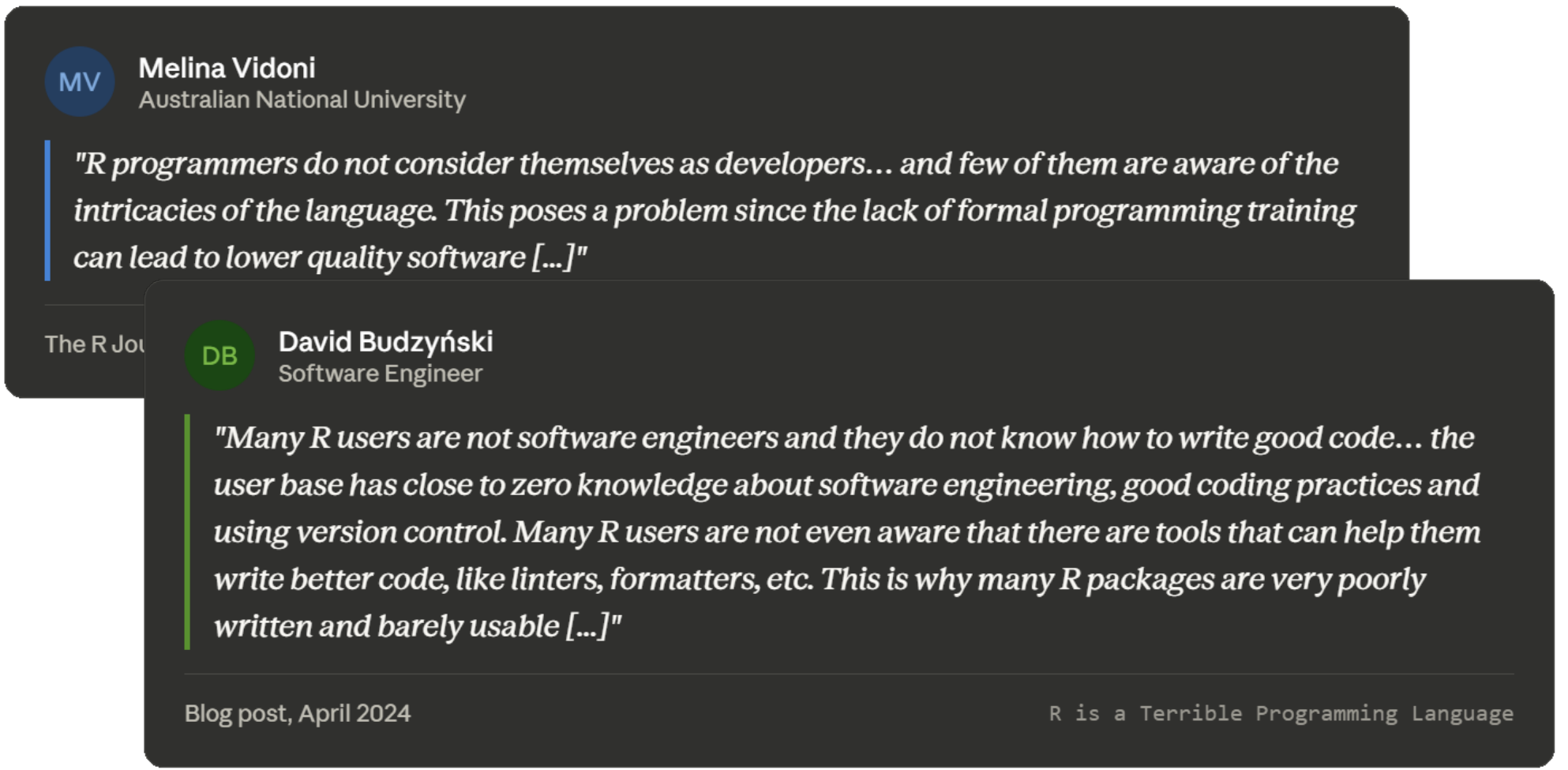
Modernising a Legacy Code Base

Experiences from Government Finance Statistics

Samuel Berger FFA

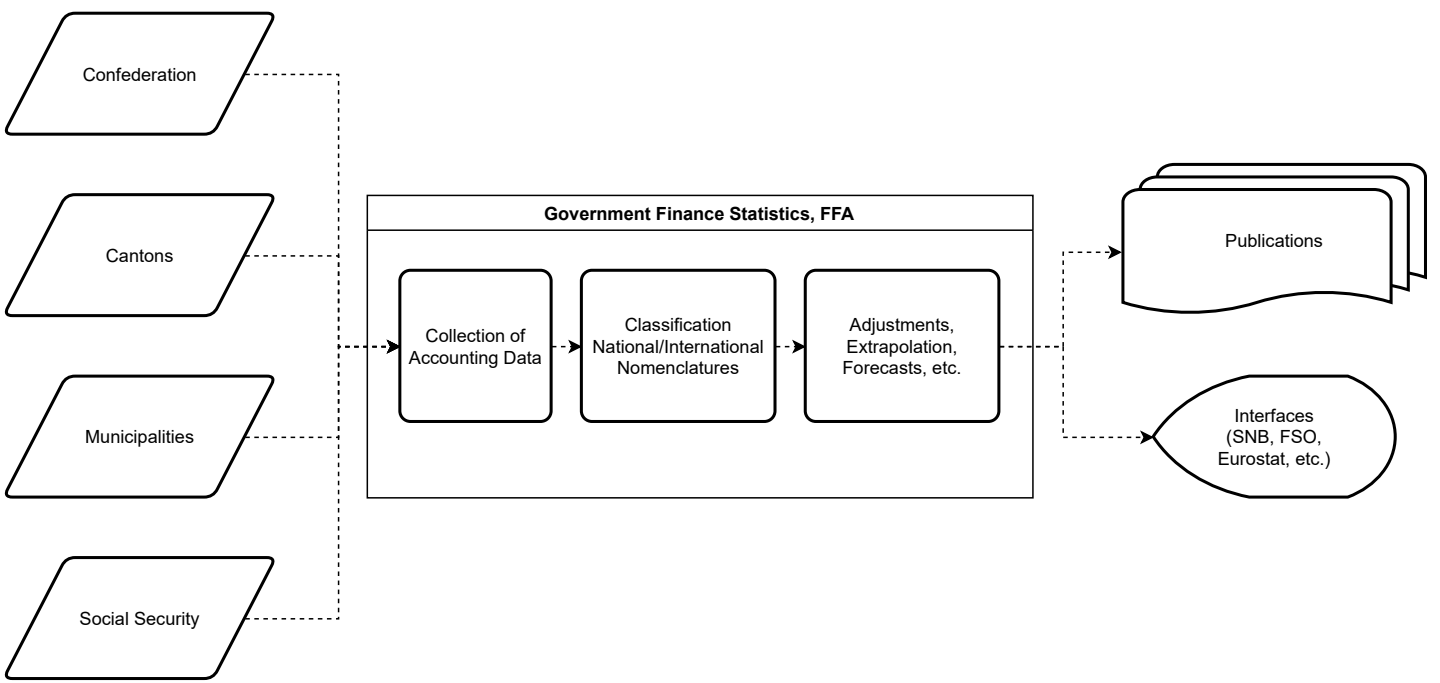
April 30, 2026

What software engineers say about R users…



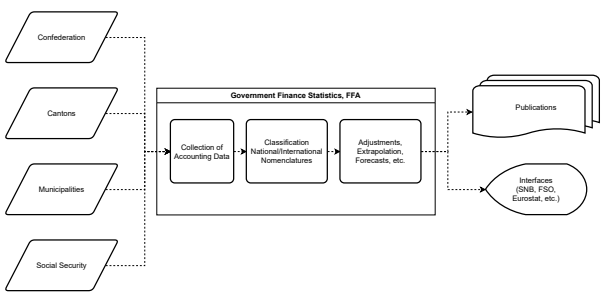
Should we care?

- About us
- R in Government Finance Statistics, FFA (*"GFS"*)



→ An extensive / complex system

R for GFS - as of 2022



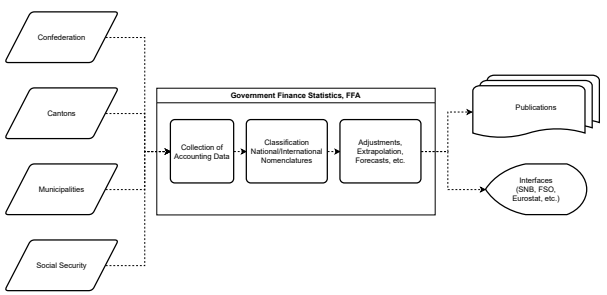
A complex system

How was it built?

- A sequence of scripts
- Heavy reliance on `.GlobalEnv`
- Data in files on a Win network share
  `.csv`, `.xml`, `.rda`
- Dev / prod on local clients
- No version control
- Built and extended over years
- Many (isolated) contributors

→ Common approaches for one-off data wrangling and interactive analysis

R for GFS - as of 2022



A complex system

How was it built?

- A sequence of scripts
- Heavy reliance on `.GlobalEnv`
- Data in files on a Win network share
  `.csv`, `.xml`, `.rda`
- Dev / prod on local clients
- No version control
- Built and extended over years
- Many (isolated) contributors

Common approaches for one-off data wrangling and interactive analysis

Why is this a problem?

Reproducibility

Output cannot be reliably traced to sources.

- Error-prone data historisation, no version control
- Re-running older runs is nearly impossible

Operability & Collaboration

A system as fragile as the individuals running it.

- Tied to local machines; onboarding is hard
- Isolated contributions drift apart over time

Maintenance & Extendability

The system can be patched, but not improved.

- Diverging pipelines erode consistency across products
- Complexity accumulates without refactoring

Risk Management

Critical knowledge is implicit.

- No version control means no way back
- Black-box components are hard to validate or replace

~~Road to Paradise~~

Guide for Small, Incremental Improvements

1. Centralise environments (Dev / Prod)
2. Introduce version control
3. Refactor your code
4. Move to a relational data model
5. Communicate (multi-directional)

Centralise Environments

R on personal clients

One environment per team member — no reproducibility.

- Potentially different R versions
- No shared package management
- FOITT: Restricted to RStudio
- Hardware resource constraints

Workbench: a centralised environment

Managed and shared environments.

- Managed R / Python environments
- Delegate configuration:
  - version control, collaboration and CI/CD platforms
- Beyond RStudio: better support for engineering workflows

But…

Workbench needs a home — and a chain of providers to support it.

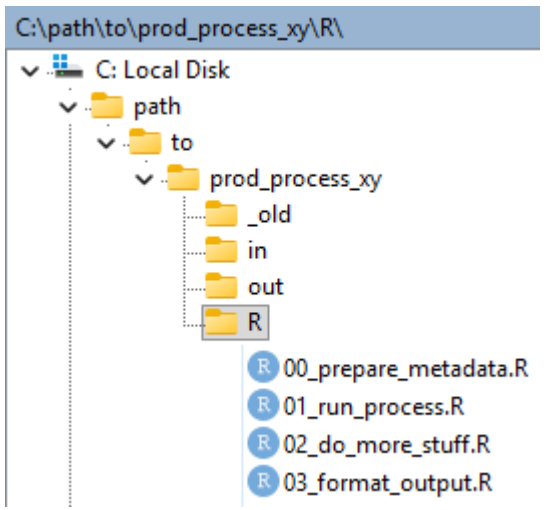
- Hosting and DevOps services

Version Control I

Starting point

The code base is not under version control.

- Unversioned scripts on a Win network share
- Loose conventions on folder/script structures
- Local code needs changes to run on WB
- Little `git` experience in the team
- Production on local machines is ongoing



Goal:

→ version the existing code

→ start moving to Workbench

→ minimise interruptions for production

Version Control II

Migration path

Incrementally move out of the file share.

1. Set up `git` (local, Workbench)
2. Set up a code hosting service
3. Create a transit repository
    - add local scripts
    - in their original folder structure
4. Check out back onto the share
    - symlinks / shortcuts
    - avoid RStudio projects
5. Branch and refactor until local code can be retired
6. Create dedicated repos:
    - per process / product
    - per production team
7. Continue refactoring and move to new repos

Refactoring I

Why refactor?

1. Local scripts will not run in the new environment
    - Path definitions
    - Database connections
    - Local utilities (funcs, packages)

Refactoring path

Get things to run again.

1. Write platform-agnostic utilities
2. Regroup scripts in repos
    - Delegate to owners of corresponding processes

Windows:

```
path_in <- "O:/local/path/to/my/subfolder/my_data.rds"
dat <- readr::read_rds(path_in)
```

Agnostic:

```
path_in <- migutils::path("my/subfolder/my_data.rds")
dat <- readr::read_rds(path_in)
```

Refactoring II

Why refactor?

2. Long scripts increase costs and risks
    - Robustness issues and difficult debugging
    - Patching instead of extending
    - Black-box processes, limited self-documentation

Refactoring path

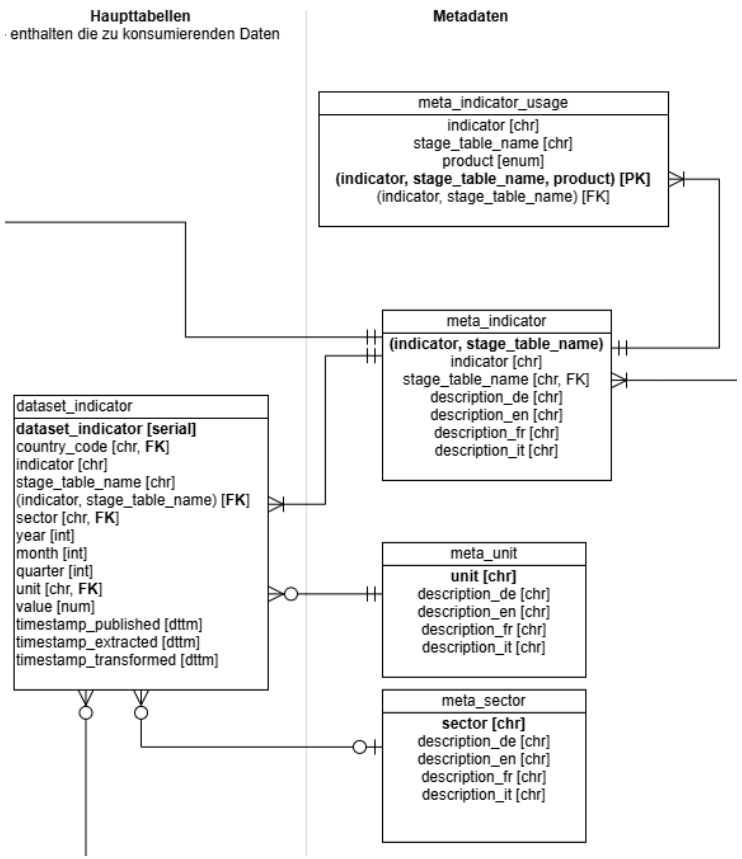
Stop scripting – start programming.

1. Slice scripts into modular functions
2. Move functions into packages
3. Read all input at the start
4. Write all output at the end
5. Iterate and compare output

Guidelines:

No production logic outside of R packages	Separate code and data	Remove cyclical dependencies	Minimise coupling with pure functions

Relational Data Modelling



What about the data?

Carefully craft a relational database model.

1. For structured data, an SQL data model
   - defines data consistency rules
   - reliably enforces them
   - allows for flexible extension
2. Iteratively integrate existing data

⇒ *Invest in conception and maintenance!*

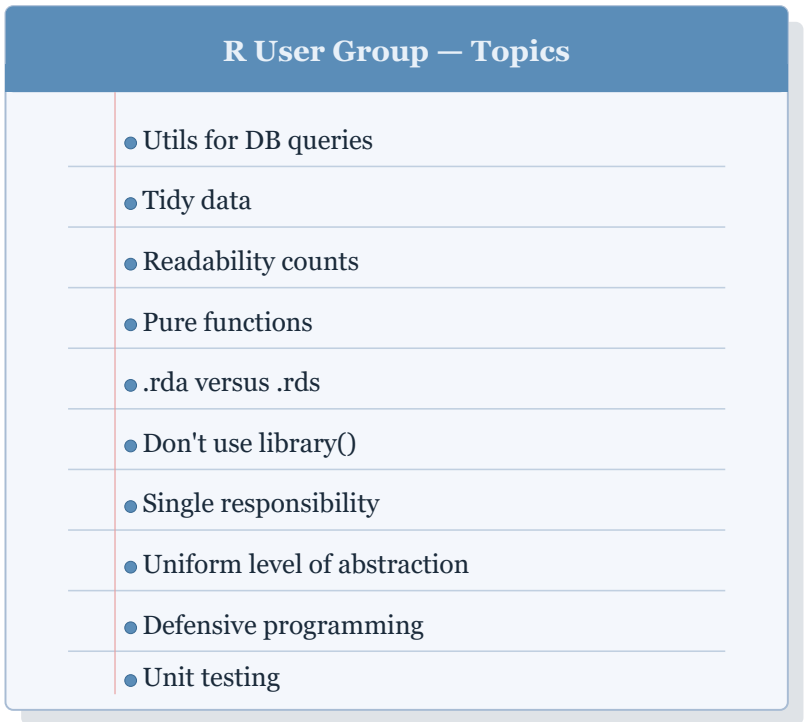
Note: Theoretical foundations

Communication

Promote common understanding

Within the team:

- Internal R user group meetings



Communication

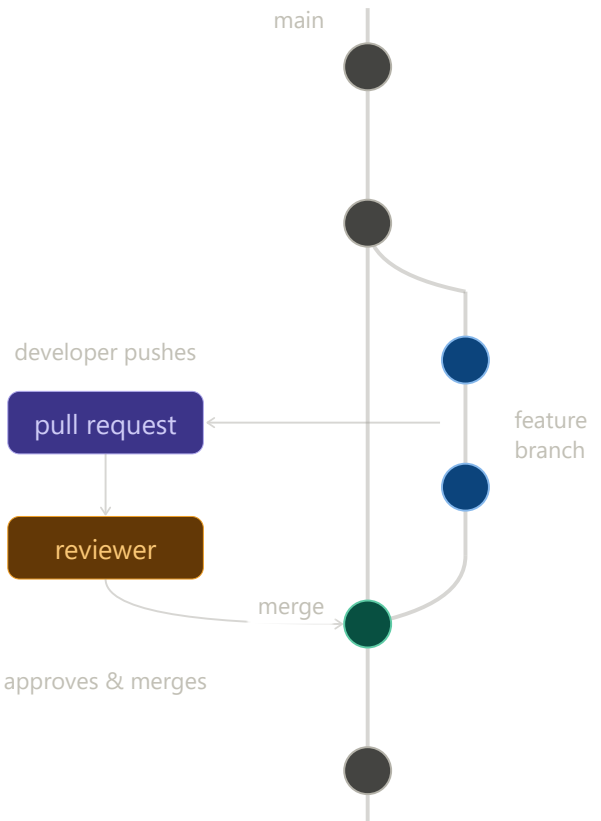
Promote common understanding

Within the team:

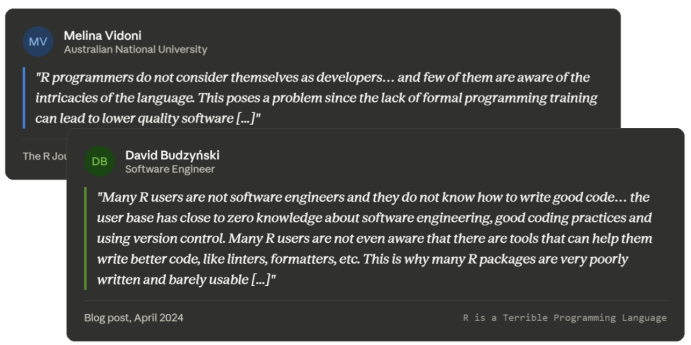
- Internal R user group meetings
- Pull requests and reviews

Towards management:

- Awareness for risks and costs of legacy code
- Benefits of refactoring and modernisation
- Balance clean-up and new feature development



What software engineers say about R users…



Do we now know how to write "good code"?

Yes — by:

- Breaking out of isolation
- Being curious about new practices
- Exchanging with others

Thank You

Questions & Discussion

Samuel Berger · samuel.berger@efv.admin.ch